ENTREGA 01 / PROJETO INTEGRADOR 3

Plano de testes

Cadastro de usuário e consulta ao catálogo

Projeto	ISAC Brechó - Bazar Solidário
Parceiro	ISAC, por meio do Porto Digital
Módulo	Cadastro e catálogo do front-end
Revisão	3.0 | 18 de outubro de 2026
GitHub	https://github.com/atlasaqui/bazar-solidario
Branch publicada	entrega/01-documento-casos-de-teste
Código publicado	https://github.com/atlasaqui/bazar-solidario/tree/entrega/01-documento-casos-de-teste

Sobre esta entrega

Este plano reúne dois casos de uso e dez casos de teste do ISAC Brechó. O cadastro foi escolhido por envolver validação de dados e comunicação com o servidor. O catálogo permite verificar busca, filtros, favoritos e ordenação das peças.

Equipe

Victor William Monteiro da Rocha
Geovana Leticia
Silas Freitas
Mateus Ferreira
Willyanne Darline
Ruan Araujo
Suellen da Silva
Cynthia Melo

Conteúdo do documento

2 casos de uso | 5 testes por caso de uso | 10 testes no total | Matriz de rastreabilidade

Os cenários incluem o uso esperado do sistema e situações em que a operação deve ser interrompida ou corrigida. Cada caso informa os dados de entrada, o procedimento e o resultado que deve ser observado.

01 Cadastro de usuário

Identificador	CU01
Nome	Cadastrar usuário
Descrição	Receber os dados do visitante, validar o formulário e apresentar a confirmação ou o erro retornado pela API.
Atores	Visitante; API de usuários como sistema de apoio.
Pré-condições	Página /cadastro disponível. Para cadastrar no servidor, a API e o banco precisam estar ativos.
Pós-condições	Em caso de resposta válida, a interface confirma o cadastro e limpa os campos. Em caso de falha, mantém o formulário disponível para correção ou nova tentativa. O cadastro não inicia uma sessão de login.

Fluxo principal

1. O visitante abre a página de cadastro.
2. Informa nome, e-mail, senha e confirmação de senha.
3. Aciona o botão Criar cadastro.
4. O sistema valida os campos e a confirmação.
5. O cliente envia a requisição POST /api/usuarios.
6. O sistema verifica o identificador e o perfil CLIENTE da resposta.
7. A interface apresenta a confirmação e limpa o formulário.

Fluxos alternativos

A1 Dados inválidos	O envio é bloqueado até que os campos sejam corrigidos.
A2 Senhas diferentes	A confirmação deve ser corrigida antes de uma requisição.
A3 Conflito de e-mail	Uma resposta HTTP 409 gera uma mensagem de e-mail já cadastrado.
A4 Falha de rede	A interface informa a indisponibilidade e permite tentar novamente.
A5 Resposta inválida	Dados fora do contrato não são apresentados como cadastro concluído.

02 Consulta ao catálogo

Identificador	CU02
Nome	Consultar peças do catálogo
Descrição	Encontrar peças por nome ou brechó, combinar filtros e organizar os resultados por preço ou curadoria.
Atores	Visitante ou usuário do aplicativo.
Pré-condições	Catálogo demonstrativo com seis peças carregado. No fluxo de favoritos, deve haver peças salvas no navegador.
Pós-condições	A listagem reflete os critérios escolhidos ou apresenta o estado sem resultados. A consulta não modifica os dados originais das peças.

Fluxo principal

1. A pessoa abre /catalogo e visualiza as peças.
2. Informa um termo de busca.
3. Seleciona categoria e tamanho, quando necessário.
4. Escolhe a ordenação por preço ou curadoria.
5. O sistema aplica os critérios e atualiza a listagem.

Fluxos alternativos

A1 Sem resultados	A listagem fica vazia e permite alterar ou limpar a busca.
A2 Favoritos	A consulta considera somente as peças salvas.
A3 Variação de escrita	Busca ignora acentos, caixa e espaços nas extremidades.
A4 Limpar critérios	A curadoria original é recuperada ao remover os filtros.

Testes CT01 e CT02

CT01 / Cadastrar com dados válidos

Identificador	CT01
Caso de uso	CU01
Objetivo	Verificar o envio de dados válidos e o tratamento de uma resposta de cadastro bem-sucedido.
Entrada	Nome: Ana; e-mail: ana@example.com; senha e confirmação: exemplo123. API simulada retorna HTTP 201, id 1 e role CLIENTE.
Procedimento	1. Abrir /cadastro. 2. Preencher os quatro campos. 3. Acionar Criar cadastro.
Resultado esperado	O cliente envia nome, e-mail e senha para /api/usuarios. Exibe confirmação com o nome e não guarda nem retorna a senha para a interface. Cadastrar não inicia uma sessão.
Urgência	Alta
Referência no código	usuarios.test.ts: cadastro respeita o contrato existente e descarta a senha da resposta

CT02 / Impedir cadastro com campos inválidos

Identificador	CT02
Caso de uso	CU01
Objetivo	Verificar se dados inválidos são recusados antes de acessar a API.
Entrada	Variações independentes: nome vazio ou só espaços; e-mail sem @; senha com menos de 8 caracteres. Manter os demais campos válidos.
Procedimento	1. Abrir /cadastro. 2. Executar cada variação separadamente. 3. Tentar enviar. 4. Conferir mensagem e ausência de requisição.
Resultado esperado	A validação nativa ou a classe informa o campo inválido; nenhum cadastro é enviado e nenhum falso sucesso aparece.
Urgência	Alta
Referência no código	UsuarioService.test.ts: valida campos antes de fazer qualquer requisição

Testes CT03 e CT04

CT03 / Impedir senhas divergentes

Identificador	CT03
Caso de uso	CU01
Objetivo	Verificar se a divergência entre senha e confirmação impede o envio.
Entrada	Ana; ana@example.com; senha exemplo123; confirmação diferente123.
Procedimento	1. Preencher o formulário com os dados. 2. Acionar Criar cadastro. 3. Observar a mensagem.
Resultado esperado	Exibe As senhas precisam ser iguais. Não faz requisição à API. Permite corrigir a confirmação e tentar novamente.
Urgência	Alta
Referência no código	UsuarioService.test.ts: rejeita confirmação diferente sem enviar dados

CT04 / Tratar e-mail já cadastrado

Identificador	CT04
Caso de uso	CU01
Objetivo	Verificar a mensagem apresentada quando a API informa conflito de e-mail.
Entrada	Dados válidos; API simulada responde HTTP 409.
Procedimento	1. Configurar a resposta 409 no teste. 2. Enviar o cadastro. 3. Conferir o erro devolvido ao formulário.
Resultado esperado	Informa Este e-mail já está cadastrado. Use outro e-mail. Não exibe sucesso nem detalhes internos do servidor.
Urgência	Alta
Referência no código	usuarios.test.ts: erro HTTP 409 não expõe detalhes internos do backend

Testes CT05 e CT06

CT05 / Tratar indisponibilidade de rede

Identificador	CT05
Caso de uso	CU01
Objetivo	Verificar se uma falha de conexão produz uma mensagem recuperável.
Entrada	Dados válidos; transporte simulado lança TypeError de conexão.
Procedimento	1. Configurar falha de rede no teste. 2. Solicitar cadastro. 3. Conferir a rejeição e, no formulário, a liberação para nova tentativa.
Resultado esperado	Informa que não foi possível conectar ao servidor. Não apresenta sucesso. O formulário encerra o estado de envio; a pessoa pode tentar novamente.
Urgência	Alta
Referência no código	usuarios.test.ts: falha de rede produz mensagem recuperável

CT06 / Listar a curadoria sem filtros

Identificador	CT06
Caso de uso	CU02
Objetivo	Conferir a quantidade e a ordem das peças na curadoria inicial.
Entrada	Catálogo ilustrativo com seis peças; busca vazia; categoria e tamanho Todos; ordenação Nossa curadoria.
Procedimento	1. Abrir /catalogo. 2. Conferir quantidade e ordem das peças.
Resultado esperado	Exibe seis achados na ordem dos IDs 1, 2, 3, 4, 5 e 6.
Urgência	Normal
Referência no código	CatalogoService.test.ts: lista todas as peças por padrão; mantém curadoria após ordenar

Testes CT07 e CT08

CT07 / Buscar sem depender de acentos ou caixa

Identificador	CT07
Caso de uso	CU02
Objetivo	Verificar a busca com variações de acentos, letras maiúsculas e espaços.
Entrada	Consulta TENIS com espaços externos; segunda consulta raizes.
Procedimento	1. Abrir /catalogo. 2. Buscar TENIS. 3. Limpar e buscar raizes.
Resultado esperado	TENIS encontra Tênis casual (ID 4). raizes encontra Camisa de linho e Jaqueta jeans (IDs 1 e 3), do Brechó Raízes.
Urgência	Normal
Referência no código	CatalogoService.test.ts: busca nome ignorando acentos, caixa e espaços externos; busca pelo brechó

CT08 / Combinar filtros e favoritos

Identificador	CT08
Caso de uso	CU02
Objetivo	Verificar a aplicação simultânea de busca, filtros e favoritos.
Entrada	Favoritos: IDs 1 e 3; categoria Roupas; tamanho G; texto jeans.
Procedimento	1. Salvar as peças 1 e 3 nos favoritos. 2. Abrir /favoritos. 3. Selecionar Roupas, tamanho G e buscar jeans.
Resultado esperado	Exibe somente Jaqueta jeans (ID 3). Todos os critérios são aplicados simultaneamente.
Urgência	Normal
Referência no código	CatalogoService.test.ts: combina categoria, tamanho, favoritos e texto

Testes CT09 e CT10

CT09 / Exibir estado vazio em busca sem resultado

Identificador	CT09
Caso de uso	CU02
Objetivo	Verificar a resposta da listagem quando a busca não encontra peças.
Entrada	Consulta inexistente.
Procedimento	1. Abrir /catalogo. 2. Digitar inexistente. 3. Observar a listagem.
Resultado esperado	Retorna zero achados e apresenta Nenhum achado por aqui. A pessoa consegue limpar ou alterar a busca.
Urgência	Normal
Referência no código	CatalogoService.test.ts: retorna vazio quando não há correspondência

CT10 / Ordenar por preço sem perder a curadoria

Identificador	CT10
Caso de uso	CU02
Objetivo	Verificar a ordenação por preço e a preservação da curadoria original.
Entrada	Seis peças, sem filtros; alternar Menor preço, Maior preço e Nossa curadoria.
Procedimento	1. Abrir /catalogo. 2. Selecionar Menor preço. 3. Selecionar Maior preço. 4. Voltar à curadoria.
Resultado esperado	Ordem crescente: 35, 58, 65, 75, 89 e 95 reais; decrescente inversa. A curadoria volta aos IDs 1 a 6 e o catálogo original não é modificado.
Urgência	Normal
Referência no código	CatalogoService.test.ts: ordena preços crescentes e decrescentes; mantém curadoria após ordenar

Rastreabilidade

Caso de uso	Casos de teste	Código relacionado
CU01 Cadastrar usuário	CT01 a CT05	UsuarioServiceTest e regressões HTTP
CU02 Consultar peças	CT06 a CT10	CatalogoServiceTest

Critérios de verificação

Os casos de teste descrevem o comportamento esperado do sistema. A suíte automatizada verifica as regras das classes e o contrato do cliente HTTP. A aparência das telas e as mensagens do formulário também foram conferidas no protótipo local.

Ambiente	Node.js 22.18 ou superior; React; TypeScript; Node Test Runner.
Execução	Na pasta frontend: npm ci, npm test e npm run test:evidencias.
Urgência Alta	Falhas que impedem ou comprometem o cadastro.
Urgência Normal	Falhas na consulta e na organização das peças.

Limite da verificação

As respostas da API são simuladas nos testes unitários. Pagamento, chatbot, login e persistência no banco não fazem parte desta verificação.